VibeCoders:
Sebastian Grammas, Peijing Xu, Alan Lee, Qizheng Sui
04/24/2026

Grocery Path Finder: A More Efficient way to Shop

AGENDA

01 What is the problem we are trying to solve?

02 What features did we implement to help solve this?

03 Development and Technical Strategies

04 Future of the Application

05 Group Dynamics

CHALLENGES

01

Grocery Shopping Taking too Long

Have you ever wandered every aisle just to find one item?

02

Find the Best Deals

Coupons are scattered across apps, flyers, and loyalty cards with no unified view.

03

Optimizing Your Budget

You can't easily compare your full cart across Walmart, Hannaford, Aldi, and Price Chopper simultaneously.

HIGHLIGHT PRODUCT BENEFITS

01

Grocery Shopping Takes Too Long

Aisle information for your items by store zone to minimize walking distance.

02

Find the Best Deals

Centralized coupon engine aggregates % off, $ off, BOGO, and multi-buy deals automatically.

03

Optimizing Your Budget

Parallel price tracking runs base, post-coupon, and savings totals across all grocery stores in the database.

01

Grocery Shopping Takes Too Long

The problem this feature will solve is that going through every store's website is tedious and takes way too long. Finding the items in a store is also an unpleasant experience during shopping.

Existing alternatives include APIs or websites created for specific stores, our product aggregates several stores into one to improve a customer's efficiency, our product shows where each item is ordering them by aisle numbers.

General features

- Shopping cart that users can populate
- Have a optimize my cart button that:
  - Allow the user to select items that they have found

02

Find the Best Deals

Coupon deals doesn't typically show up on retailer websites and can be quite tedious to sort and calculate the final sum of money needed to buy products after deducting special deals and coupons.

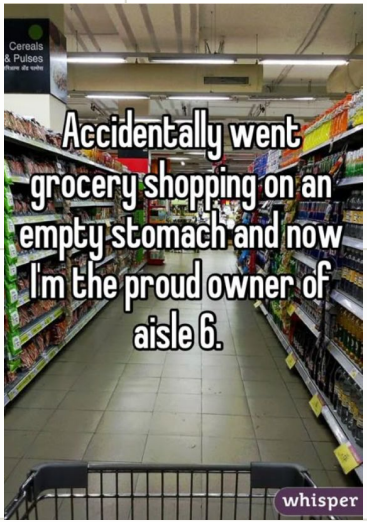
This feature help combine all of the special deals together such that once a user creates a shopping list, the service would automatically detect and deduct the discount.

General features

- In the page for each store there are several special and coupon deals available.
- Selecting items under the special deal section and clicking the optimize my cart button will:
  - Automatically take into account the deals and detect from the total price.

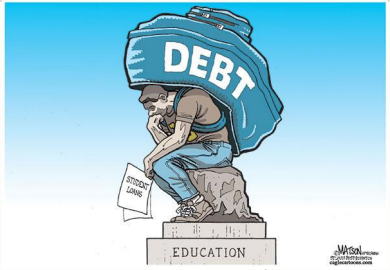
03

Optimizing Your Budget



The problem with online shopping is that most people prefer the cheapest options. There is no easy way to compare prices between multiple stores and customers will have to just go to each to find out if they are getting the most optimal deals

The Grocery finder is able to compare a shopping cart full of items with all other stores that our service covers. This improves the user's quality of life and provides relevant information to them on whether or not they are overpaying on certain products



General features

- Populate a shopping cart for a specific store and click on optimize my cart button
- Once on the next page select the option to compare with other stores
  - Will show the user the price comparison of the items in the shopping cartl

Development and Technical Strategy (Patterns)

01

Factory Method Pattern

AdvancedMatcherFactory evaluates context top-down:
coupon mode → budget → default.

02

Strategy Pattern

Two interchangeable ProductMatchers(CouponAware, BudgetAware) and an AislePlanner (Default, sorts by zone_order).

03

Service Layer

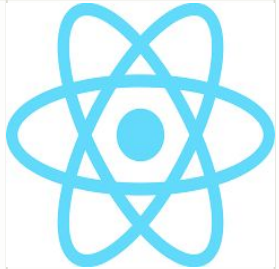
Full pipeline is owned by one script as a single orchestrator: build context → select factory → match products → apply coupons → plan route → calculate savings

Development and Technical Strategy (Implementation)

01

Tech Stack

React 18 + TypeScript + Tailwind + Radix UI + Figma on the frontend. Flask + SQLAlchemy + SQLite + JWT on the backend. Vite for building tool.



02

API Design

Flask routes split into modular Blueprints: auth, stores, items, coupons, cart, optimize. Each Blueprint is independently testable and extendable.

03

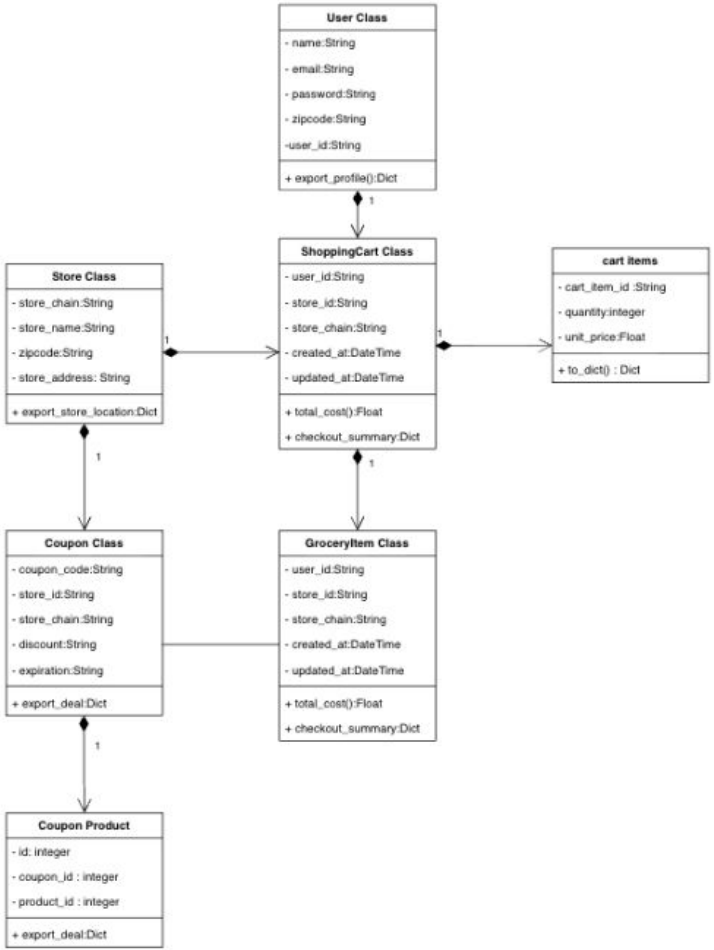
Data & Algorithms

Static product/coupon DB seeded per store chain from KrogerAPI. Fuzzy matching with exact → substring → keyword fallback. Three parallel price totals tracked throughout: base, post-coupon, and savings delta.

System Arcitecture

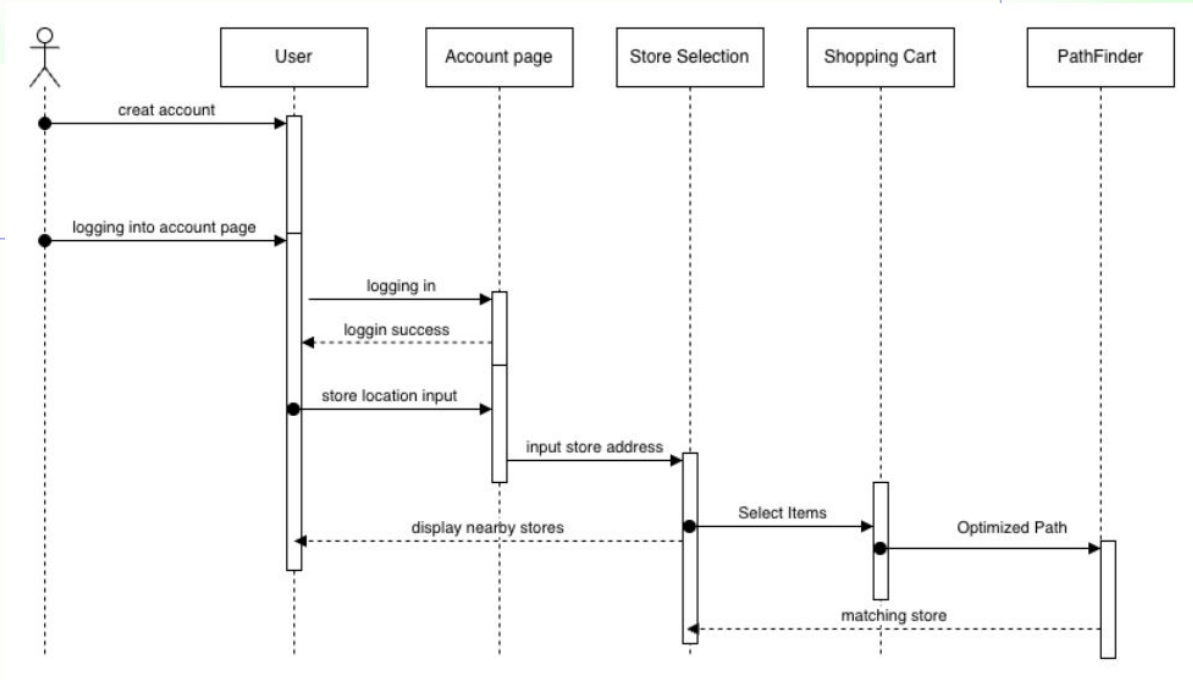
Data Model

System is built around a central ShoppingCart owned by a User, which aggregates CartItems and GroceryItems. Stores and Coupons are first-class entities and a Coupon belongs to a store. This structure allows the three parallel price totals to be computed cleanly at checkout.

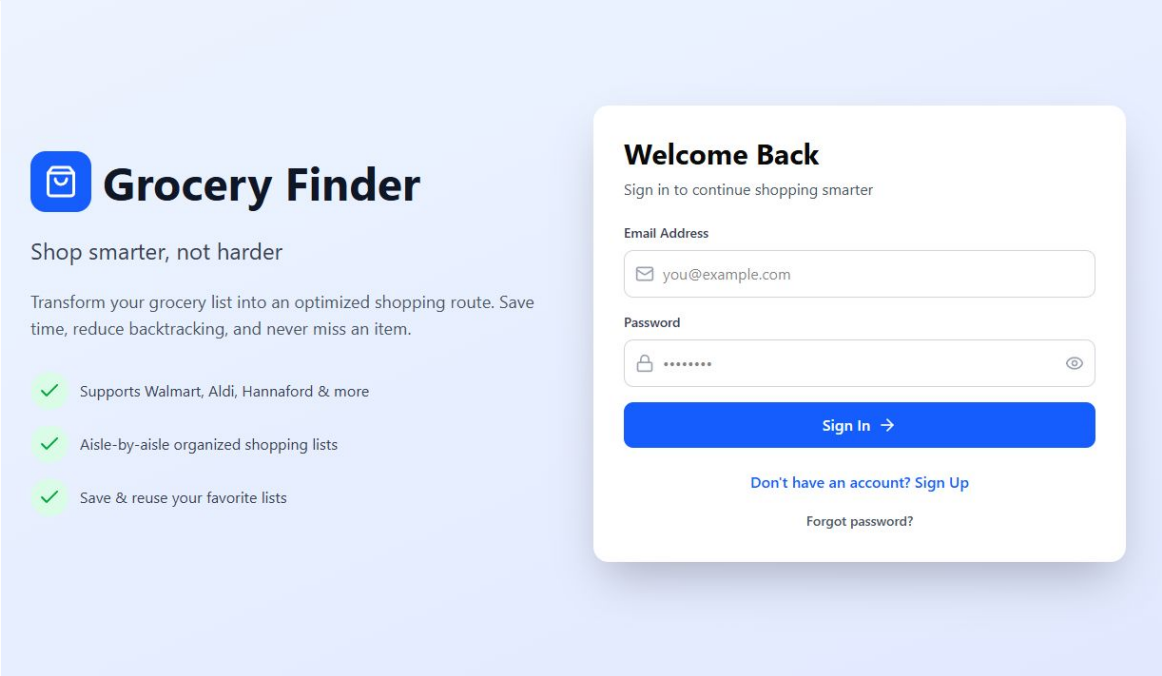


User Flow & Sequence

After authenticating, the user inputs their store location, which returns nearby stores. From there they select items in their cart, which triggers PathFinder. The optimize pipeline runs and returns both the optimized aisle route and the matching store. Everything downstream of login is a single linear flow keeping the backend API design straightforward.

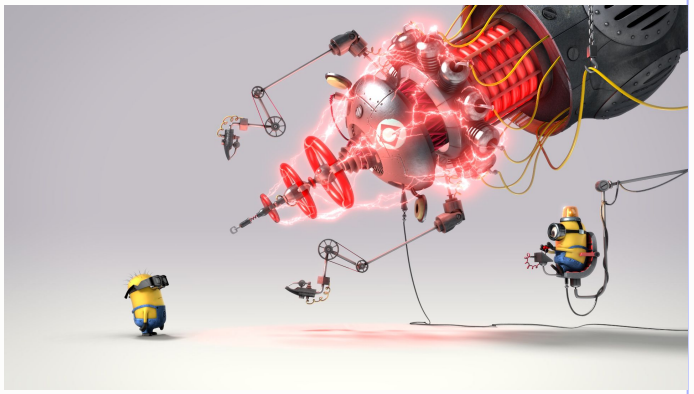


FINAL PRODUCT LIVE DEMO

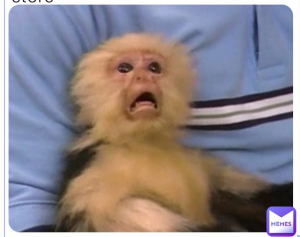


Beta Testing

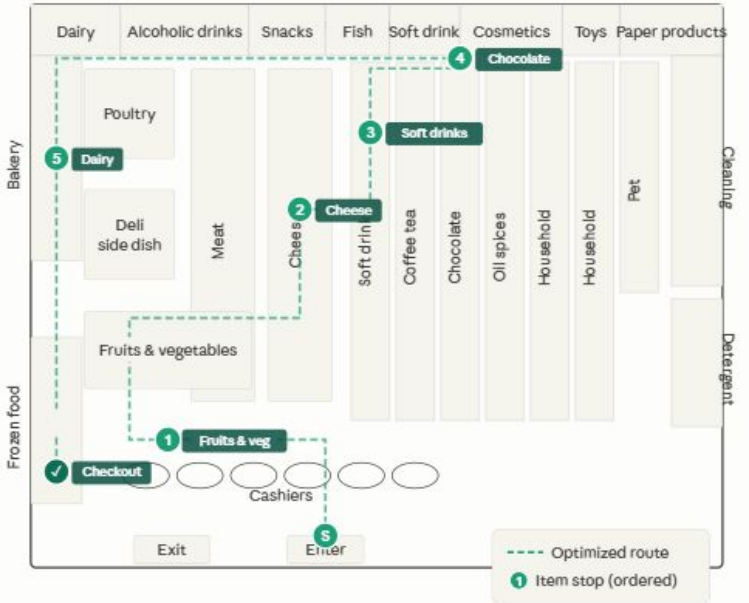
- Testers were given tasks to test authentication flow, core features, and changes to accessibility settings
- All tests were passed, but a minor issue was found relating to coupon logic (Fixed after)
- Functionality worked as expected by the User
- Testers found the overall process flow to be intuitive and simple to understand



Me when i get lost in a grocery store



Future of Grocery Path Finder



In-Store Optimized Path Navigation

Move beyond aisle ordering to true visual path using real store map data, the app would guide users along the most efficient physical route through the store, eliminating backtracking entirely.

Live Store Inventory

Currently, the product DB is seeded with static data. The v2.0 goal is to connect to live inventory APIs so item availability and pricing reflects what's actuall on the shelves at the store that day

Real Store Discovery via Google Maps API

Right now supported stores are hardcoded for the Troy area. Integrating the Google Maps or Google Places API would let users find any nearby grocery store by location, making the app work anywhere rather than just the stores we manually configured.

Store Brand Preference Filter

Allow users to set a preference for store brand alternatives. When optimizing, the matcher would automatically suggest the cheaper store brand equivalent for any name brand item in the cart, maximizing savings without the user having to manually swap items

Project Retro

What worked?

We had a clear definition of what we wanted to get done at the beginning of each sprint. Weekly standups along with the time we met in class and our weekly meeting with TA kept us on track.

What didn't work?

We had too ambitious of a scope. We thought that we would be able to get live inventory and store layouts but that just wasn't the case so we had to pivot to hardcoding and seeding data solely from KrogerAPI and manually adjusting prices so that each store did not have the same prices.

What would we keep?

Our communication was solid and we were able to get through adversity of group members not having computers.

We were still able to all make meaningful contributions and stay on top of our work despite this.

What would we change?

The biggest issue we ran into was multiple people working on DB at the same time. This created merge conflicts that were not easy to resolve. Would change to assigning only one person to work on the DB at a time.

Challenges Faced

TECHNICAL CHALLENGES

The team encountered some technical problems during the beginning stages of development, the team originally sought APIs from specific stores and will attempt to integrate them all into one. However, this approach failed because most APIs found are paid, and the quantity of usage by our product will far exceed our team's budget.

TEAM & WORKFLOW CHALLENGES

The most significant challenge faced by the team would be losing a team member: Originally we were a team of five people, but due to medical emergency, one member of the team had to leave. This had a significant impact on the amount of work the team was able to accomplish.

HOW WE ADDRESSED THEM

The team addressed the technical and non technical challenges faced by the team by redoing the project plan to make up for the loss of a team member, and as a group we decided to scrap from the Kroger API and then generate a dataset from the scraped results to create a proof of concept for the Grocery Finder

Group Dynamics

Working as One

In the early stages of development and planning, the team aligned quickly on system architecture and project goals without any major disputes. Having a clearly structured vision from the start meant everyone was building toward the same thing, which kept early momentum strong and avoided the kind of misalignment that tends to slow teams down mid-sprint.

Collaborative Debugging

Because all stores, inventory, and coupons were seeded statically rather than pulled from live APIs, bugs were often invisible until a very specific combination of items, stores, and coupon types was tested manually. We did this as a team in a mob programming like setup where one person would fix the issues while the other three were testing on separate devices at the same time.

WHAT WE GAINED FROM THE EXPERIENCE

The team gained valuable experience collaborating in an Agile environment, working together helped build confidence, and develop an understanding of full stack development.

Group Takeaways

Design Patterns Have Real Value

Going into the course, patterns felt theoretical. Building Factory Method and Strategy into an actual feature pipeline made the development process feel less overwhelming or more structured than previous projects we have worked on. Adding a new matcher was easy to implement without touching existing code.

Document Discipline Shapes Better Code

Writing CRC cards, UML diagrams, and requirements before implementation forced us to think about interfaces and responsibilities upfront. The parts of the codebase we documented thoroughly before building were noticeable cleaner than the parts we figured out as we went.

Individual Takeaways

Sebastian Grammas	Peijing Xu
This was the first time I have been actively involved in the planning of a project in an Agile environment. Learning how to plan and budget time in sprints was more difficult than I expected and was a good experience before entering the workforce.	I experienced what it is like to work on a project in an agile environment for the first time. Understood how a software development team functions and progresses, finally learned to tackled problems with team building and work distribution.

Individual Takeaways

Alan Lee	Qizheng Sui
This course was completely different from other courses and exposed me to full stack development. The courses I have taken are more solo assignment based as opposed to being a semester long project.	This project helped me understand the importance of planning, teamwork, and communication. I learned that Agile requires both flexibility and organization, and this experience gave me a better idea of how team projects work in a professional setting.

THANK YOU & QUESTIONS?